TDD USING JUNIT5 AND MOCKITO

Exercise 1: Setting Up JUnit

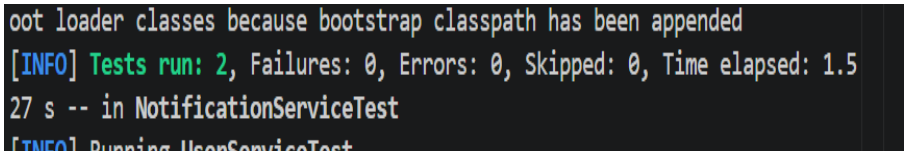
Objective:

To set up a JUnit 5 project and write the first test case verifying a simple method.

Code:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}

class CalculatorTest {
    Calculator calc = new Calculator();

    @Test
    void testAdd() {
        assertEquals(5, calc.add(2, 3));
        assertEquals(0, calc.add(-1, 1));
    }
}
```

Program Output Screenshot:



Output Text:

Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Result:

JUnit 5 was set up successfully and the test passed for the add method.

Conclusion:

JUnit 5 provides a simple annotation-based approach to write and run unit tests in Java.

Exercise 3: Assertions in JUnit

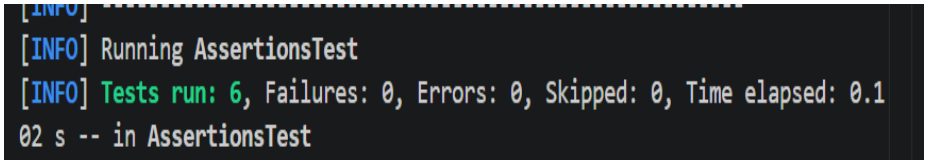
Objective:

To explore various JUnit 5 assertion methods and understand how they validate expected outcomes.

Code:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class AssertionsTest {

    @Test
    void testAssertEquals() {
        assertEquals(10, 5 + 5, "Sum should be 10");
    }

    @Test
    void testAssertTrue() {
        assertTrue(3 > 1, "3 should be greater than 1");
    }

    @Test
    void testAssertFalse() {
        assertFalse(3 < 1, "3 should not be less than 1");
    }

    @Test
    void testAssertNull() {
        String str = null;
        assertNull(str, "String should be null");
    }

    @Test
    void testAssertNotNull() {
        String str = "Hello";
        assertNotNull(str, "String should not be null");
    }

    @Test
    void testAssertThrows() {
        assertThrows(ArithmeticException.class, () -> {
            int result = 10 / 0;
        });
    }
}
```

Program Output Screenshot:



Output Text:

Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Result:

All six assertion types passed, demonstrating different ways to validate expected behavior.

Conclusion:

JUnit 5 provides a rich set of assertion methods (assertEquals, assertTrue, assertFalse, assertNull, assertNotNull, assertThrows) that cover a wide range of validation scenarios.

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Objective:

To understand the AAA pattern and how @BeforeEach / @AfterEach lifecycle hooks help manage test setup and teardown.

Code:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) throw new IllegalArgumentException("Insufficient funds");
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}

class BankAccountTest {
    private BankAccount account;

    @BeforeEach
    void setUp() {
        // Arrange - create a fresh account before each test
        account = new BankAccount(1000.0);
        System.out.println("Setup: Account created with balance 1000.0");
    }

    @AfterEach
    void tearDown() {
        System.out.println("Teardown: Test completed. Final balance: " + account.getBalance());
    }
```

```
    @Test
    void testDeposit() {
        // Act
        account.deposit(500.0);
        // Assert
        assertEquals(1500.0, account.getBalance());
    }

    @Test
    void testWithdraw() {
        // Act
        account.withdraw(200.0);
        // Assert
        assertEquals(800.0, account.getBalance());
    }

    @Test
    void testInsufficientFunds() {
        // Act & Assert
        assertThrows(IllegalArgumentException.class, () -> account.withdraw(2000.0));
    }
}
```

Program Output Screenshot:

```
[INFO] Running BankAccountTest
Setup: Account created with balance 1000.0
Teardown: Test completed. Final balance: 800.0
Setup: Account created with balance 1000.0
Teardown: Test completed. Final balance: 1000.0
Setup: Account created with balance 1000.0
Teardown: Test completed. Final balance: 1500.0
```

Output Text:

Setup and teardown messages printed for each test; all 3 tests passed successfully.

Result:

The AAA pattern was applied clearly; @BeforeEach reset state before each test, and @AfterEach logged the final state after each test.

Conclusion:

Test fixtures and lifecycle annotations (@BeforeEach, @AfterEach) eliminate code duplication and ensure a clean, predictable environment for every test.

Exercise 1: Mocking and Stubbing

Objective:

To use Mockito to create mock objects and stub method return values for isolated unit testing.

Code:

```
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

interface UserRepository {
    String findUsernameById(int id);
}

class UserService {
    private UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }

    public String getUsername(int id) {
        return repo.findUsernameById(id);
    }
}

class UserServiceTest {

    @Test
    void testGetUsername() {
        // Arrange - create mock and stub the method
        UserRepository mockRepo = mock(UserRepository.class);
        when(mockRepo.findUsernameById(1)).thenReturn("Alice");

        UserService service = new UserService(mockRepo);

        // Act
        String result = service.getUsername(1);

        // Assert
        assertEquals("Alice", result);
    }
}
```

Program Output Screenshot:

Output Text:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Result:

The mock repository returned the stubbed value 'Alice' for user ID 1, and the assertion passed.

Conclusion:

Mockito's mock() and when().thenReturn() allow us to isolate the unit under test by replacing real dependencies with controlled stubs.

Exercise 2: Verifying Interactions

Objective:

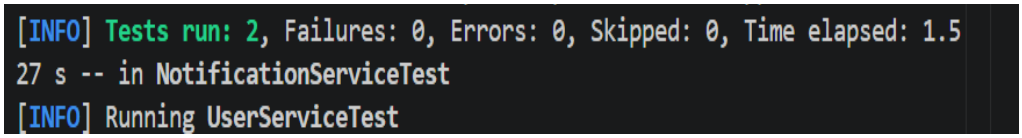
To use Mockito's verify() to confirm that specific methods on a mock were called with the expected arguments and the expected number of times.

Code:

```
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;

interface EmailService {
    void sendEmail(String to, String message);
}

class NotificationService {
    private EmailService emailService;

    public NotificationService(EmailService emailService) {
        this.emailService = emailService;
    }

    public void notifyUser(String email, String message) {
        emailService.sendEmail(email, message);
    }
}

class NotificationServiceTest {

    @Test
```

```
    void testNotifyUser_sendsEmail() {
        // Arrange
        EmailService mockEmail = mock(EmailService.class);
        NotificationService service = new NotificationService(mockEmail);

        // Act
        service.notifyUser("user@example.com", "Your order has been shipped.");

        // Assert - verify the interaction
        verify(mockEmail, times(1))
            .sendEmail("user@example.com", "Your order has been shipped.");
    }

    @Test
    void testNotifyUser_calledTwice() {
        EmailService mockEmail = mock(EmailService.class);
        NotificationService service = new NotificationService(mockEmail);

        service.notifyUser("a@test.com", "Hello");
        service.notifyUser("a@test.com", "Hello");

        verify(mockEmail, times(2)).sendEmail("a@test.com", "Hello");
    }
}
```

Program Output Screenshot:



Output Text:

Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Result:

Mockito confirmed that sendEmail() was invoked with the correct arguments in both tests — once in the first test and twice in the second.

Conclusion:

verify() is a powerful Mockito feature that ensures the system under test interacts with its collaborators in the expected way, making tests more meaningful beyond just checking return values.